Retrospective: Coroutine Executors and P2464R0

Document Number:	P4096R0
Date:	2026-03-14
Audience:	LEWG, SG1

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure
   - P2464R0
   - P2300R10
2. The Two Framings
3. The Three Criteria
4. The Error Channel
   - 4.1 The Lost Framing
   - 4.2 The Coroutine Model
   - 4.3 The Timeline
5. Lifecycle
6. Composition
   - 6.1 The Concession
   - 6.2 A Second Axis
   - 6.3 The Two Framings in Code
   - 6.4 Summary
7. Structured Concurrency
8. The Executor Is Application Policy
9. Five Years Later
   - 9.1 The Empirical Record

9.2 The Symmetry Test

9.3 Summary

Acknowledgments

References

Abstract

The paper that set aside the Networking TS analyzed the renamed API under one framing. The original framing produces different conclusions.

P2464R0[1], "Ruminations on networking and executors," identified three properties of P0443R14[3], "A Unified Executors Proposal for C++," executors: no error channel, no lifecycle for submitted work, and no generic composition. P4094R0[7], "Retrospective: The Unification of Executors and P0443," documents the scope expansion and terminology shift that produced P0443R14[3] from three independent executor models and defines two framings of `execute(F&&)` - the work framing and the continuation framing. This paper applies those framings. It documents the three properties as applied to the coroutine-native executor described in P4003R0[2], "Coroutines for I/O," and re-examines them as applied to P0443R14[3]'s `execute(F&&)`. The coroutine executor constrains the handle type to `coroutine_handle<>`, restoring the type constraint that the rename to `execute(F&&)` removed. The coroutine executor concept did not exist in its current form until P4003R0[2] was published in 2026. Sections 3-6 place P2464R0[1]'s criteria next to the coroutine executor's properties. Sections 7-8 document structured concurrency and the executor concept. Section 9 places P2464R0[1]'s predictions next to five years of published outcomes.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

1. Disclosure

This paper is part of the Network Endeavor (P4100R0), a thirteen-paper project to bring networking to C++29 using a coroutine-native approach. The authors developed and maintain Corosio[5] and Capy[4] and believe coroutine-native I/O is the correct foundation for networking in C++. The authors provide information, ask nothing, and serve at the pleasure of the chair.

The committee has been trying to standardize networking since 2005. This retrospective examines the published record to identify the failure modes that prevented delivery, so the next attempt can avoid them. Its findings stand on their own. That effort requires re-examining consequential papers, including papers written by people the authors respect.

P2464R0

P2464R0[1] was consequential. It provided the analytical framework that led the committee to set aside the Networking TS and focus on the sender/receiver model. Decisions of that magnitude deserve periodic review. This paper provides one. The intent is to ensure the committee's record is complete, not to assign blame.

P2300R10

P2464R0[1] critiqued P0443R14[3]. P2300R10[6], "std::execution," addressed those deficiencies with the sender/receiver model. P4003R0[2] provides a second async model. The committee now has two implementations to evaluate against P2464R0[1]'s criteria. This paper evaluates P4003R0[2] and uses P2300R10[6] as a reference. Structural comparisons are observations, not arguments that `std::execution` should be modified or removed.

---

2. The Two Framings

P4094R0[7] documents the scope expansion that produced P0443R14[3] from three independent executor models and the terminology shift that erased the continuation framing from the API surface. This section restates the two framings defined in P4094R0[7] Sections 2 and 6. The rest of this paper applies them.

The continuation framing. `dispatch` / `post` / `defer` schedule a continuation on an execution context. The callable is a resumption handle. The operating system performs the work. The result is delivered to the continuation when it wakes up. The executor never touches the result.

The work framing. `execute(F&&)` submits work. The callable is a unit of work. The executor runs it. If dropped, the work and its result are lost. Error handling, lifecycle, and composition are the executor's responsibility.

The structural difference is in what happens to the caller. `post` ends the caller's chain of execution. The caller returns. There is no live caller on the other end to receive a report. The continuation will be resumed later, on a context, and the result will be delivered ***to*** the continuation when it wakes up. Under the work framing, `execute` is a fork: the caller submits work and continues. The caller is alive, running, and expects to learn what happened. A live caller needs an error channel. A caller that has returned does not.

The work framing imposes requirements on the executor that the continuation framing does not. Under the work framing, the executor is responsible for error channels, lifecycle management, and generic composition - the three deficiencies P2464R0[1] identified. Under the continuation framing, the executor schedules a resumption, the OS performs the work, and the result is delivered to the continuation when it wakes up. A continuation carries no result because the result does not yet exist. The generalization from `dispatch(handler)` to `execute(F&&)` introduced these requirements. The coroutine executor's `dispatch(coroutine_handle<>)` and `post(coroutine_handle<>)` constrain the argument to a type with exactly two operations - `resume()` and `destroy()`. The continuation framing is enforced by the type system, not by naming convention.

Both framings are applied to P2464R0[1]'s three criteria below.

---

3. The Three Criteria

P0443R14[3]:

```
void execute(F&& f);
```

Coroutine executor (P4003R0[2]):

```
std::coroutine_handle<> dispatch(
    std::coroutine_handle<> h) const;

void post(std::coroutine_handle<> h) const;
```

P2464R0[1] identified three deficiencies in P0443R14[3] that made it inadequate as a foundation for async programming:

1. No error channel.
2. No lifecycle for submitted work.
3. No generic composition.

P2300R10[6] addressed all three with the sender/receiver model. The coroutine executor concept did not exist in 2021. P2464R0[1] could not have evaluated it. The point of the following sections is not that P2464R0[1] was wrong about P0443R14[3] - it is that the deficiencies it identified are properties of the `execute(F&&)` API

surface, not inherent properties of executor-based async. The coroutine executor demonstrates this by constraining the argument type to `coroutine_handle<>`, which eliminates the conditions that create those deficiencies.

---

4. The Error Channel

P2464R0[1] argued that P0443R14[3]'s `execute(F&&)` has no error channel at the call site, and that errors after the call are an "irrecoverable data loss." This section addresses both claims.

4.1 The Lost Framing

P2464R0[1]:

> *"The only thing that knows whether the work succeeded is the same facility that accepted the work submission."*

Three terms. All three are downstream consequences of the work framing that the rename created. Under the continuation framing, each term describes something different.

1. "The work." Under the continuation framing, there is no work. The operating system performs the I/O. The callable is a continuation that resumes after the OS completes. It is not work. It is the opposite of work - it is the thing that was suspended while the work happened.
2. "Succeeded." Under the continuation framing, a continuation does not succeed or fail. It resumes. The I/O result - error code, byte count - is delivered to the continuation when it wakes up. The continuation did not produce the result. The OS did.
3. "Accepted the work submission." Under the continuation framing, no work was submitted. The caller yielded control. Kohlhoff's original API named this `dispatch`, `post`, and `defer` - continuation-scheduling primitives (P4094R0[7] Section 6). The rename to `execute` changed the name. The operation remained the same.

P2464R0[1] analyzed the renamed API under the work framing. The use cases it describes - queue full, executor shutting down - presuppose a fire-and-forget model where a caller hands off work and moves on. The original API did not operate that way.

The coroutine executor's `dispatch` and `post` take `coroutine_handle<>`, not `F&&`.

4.2 The Coroutine Model

`co_await` suspends the caller. The caller is not running. It does not need an error channel back because it is not alive to act on one. It will either be resumed with a result or destroyed, propagating cancellation. There is no state where the caller is alive, running, and uninformed. `coroutine_handle<>` is a suspended continuation, not a unit of work.

4.3 The Timeline

P2464R0[1] said errors after submission are an "irrecoverable data loss." The error-channel question has a timeline. What exists at each stage determines what can be lost.

Trace the path of an asynchronous read in the Networking TS:

1. The caller initiates `async_read(socket, buffer, handler)`. The implementation stores the handler - a completion token - and begins the I/O. The handler means: this is how you resume me.
2. The operating system performs the read. The caller is not running. The handler is not running. The OS is doing the work.
3. The OS completes. The implementation now holds the result: an error code and a byte count. It calls `executor.dispatch(bound_handler)`, where `bound_handler` carries the result and the original handler together. In the committee's renamed API, this became `executor.execute(bound_handler)`.
4. The executor resumes the handler on the correct execution context. The handler receives the error code and the byte count.

At steps 1 and 2, there is no error information. The I/O has not completed. The operating system has not returned a result. There is no error code. There is no byte count. The only thing being passed to the executor is a resumption handle: this is how you resume me. That is all a callback is. That is all a future is. That is all a completion token is. That is all a `coroutine_handle<>` is. If the executor drops the handle at this stage, the handle carries no error information. The handle is a resumption. P2464R0[1] described information loss. At this stage, the handle contains no information. It contains a resumption.

At step 3, the result exists. The OS has completed. The result is bound into the handler before the executor sees it. The executor's job is to resume the handler on the right context. The result travels inside the handler, not alongside it. If the executor drops the bound handler at this stage, the result is destroyed with the handler. This is the strongest form of P2464R0[1]'s concern, and it is real: a result that exists inside a dropped continuation is lost.

The question is what happens to the program.

Networking TS:

```
async_read(socket, buffer,
    [](error_code ec, size_t n) {
        process(ec, n);
    });
// caller returns here - gone
```

The caller returned at the call site. If the executor drops the bound handler, a future action does not happen. The result was created by the OS, bound into the handler, and destroyed with the handler. It was never delivered to anyone. The caller is gone. The program is not hung.

Coroutine model:

```
auto [ec, n] =
    co_await socket.read_some(buffer);
// coroutine is suspended
```

The coroutine is suspended. The ownership contract requires the executor to resume or destroy. If it destroys the handle, the coroutine frame is destroyed, the parent is notified via destruction propagation. The result was never delivered. The coroutine was suspended at the time of destruction. The program is not hung.

Model	Initiator state	Result	Program state
Networking TS	Gone (returned)	Destroyed with handler	Not hung
Coroutine model	Suspended	Destroyed with operation state	Not hung (ownership contract)

In both models, the initiator is not running at the time of the drop. In the Networking TS, the initiator returned. In the coroutine model, the initiator is suspended. In both models, a dropped continuation destroys the result it carries. The difference between the two framings is not whether the result can be lost - it can - but whether the program hangs. Under the work framing, a dropped callable with no error channel leaves the program in an indeterminate state. Under the continuation framing, the caller is either gone (returned) or governed by an ownership contract (resume or destroy). The program state is determinate in both cases.

`sync_wait` is a testing and bridging utility, not a production async pattern. This comparison is limited to models where the initiator is asynchronous.

---

5. Lifecycle

P2464R0[1]:

> *"A P0443 executor is not an executor. It's a work-submitter."*

Under the continuation framing, the callback is a continuation - it is how the initiator resumes after the OS completes. The executor takes it and resumes it on a context.

> *"Trying to entertain the fantasy that the work submission and result observation are separable concepts is just **wrong**."*

Under the continuation framing, work submission and result observation are not separable because they are not separate things. The executor takes a continuation and resumes it on a context. The work is done by the operating system. The result is delivered ***to*** the continuation when it resumes - result observation is not separable from the continuation because the result arrives inside it.

The coroutine model makes the lifecycle explicit.

`post(coroutine_handle<>)` has two call-site dispositions:

- **Normal return.** Ownership transferred. The executor holds the handle and is responsible for it.
- **Exception.** Scheduling rejected. The caller still owns the handle.

After ownership is transferred, the execution context has two dispositions:

- **Resume.** The continuation runs.
- **Destroy without resuming.** The awaiting coroutine is also destroyed, propagating cancellation upward through the coroutine tree. This is analogous to `set_stopped` in P2300R10[6].

The executor holds a typed resource with a defined lifecycle. A `coroutine_handle<>` has exactly two operations: `resume()` and `destroy()`. The proposed wording specifies the ownership contract:

> ```
> std::coroutine_handle<> dispatch(
>  std::coroutine_handle<> h) const;
>
> void post(std::coroutine_handle<> h) const;
> ```
>
> ***Preconditions:*** `h` is a valid, suspended coroutine handle.

> ***Effects:*** Arranges for `h` to be resumed on the associated execution context. For `dispatch`, the implementation may instead return `h`. For `post`, the coroutine is not resumed on the calling thread before `post` returns.
>
> ***Postconditions:*** For `post`, ownership of `h` is transferred to the implementation. For `dispatch`, if `noop_coroutine()` is returned, ownership of `h` is transferred to the implementation; if `h` is returned, ownership of `h` is transferred to the caller via the return value and the implementation has no further obligation regarding `h`. When ownership is transferred to the implementation, the implementation shall either resume `h` or destroy `h`. No other disposition is permitted.
>
> ***Returns (*** *`dispatch`* ***only):*** `h` or `noop_coroutine()`.
>
> ***Throws:*** `std::system_error` if scheduling fails. If an exception is thrown, ownership of `h` is not transferred; the caller retains ownership.
>
> ***[Note:*** The returned handle enables symmetric transfer. The caller's `await_suspend` may return it directly. ***- end note]***

The ownership contract specifies two dispositions: resume or destroy. The `Executor` concept definition (Section 8) checks for the existence of `dispatch` and `post` but cannot enforce the ownership semantics at compile time. This is a semantic requirement, not a syntactic one - the same class of constraint that `Allocator`, `Iterator`, and every other standard library concept carries alongside its syntactic checks.

---

6. Composition

P2464R0[1]:

> *"Composing Networking TS completion handlers or asynchronous operations (which aren't even a thing in the NetTS APIs, so how would I be able to compose those other than in an ad-hoc manner?) seems like an ad-hoc and non-generic exercise. Whether they were designed for or ever deployed in larger-scale compositions that scale from low-level to high-level with any sort of uniformity, I don't know."*

6.1 The Concession

On the axis of algorithmic composition - `retry`, `when_all`, `upon_error`, chaining operations generically into higher-level workflows - the Networking TS had no generic mechanism. Each composed operation required a hand-written state machine. Boost.Beast (Boost 1.66, 2017) deployed three layers of composed asynchronous

operations - socket reads into HTTP parsing into WebSocket framing - and every layer required its own state machine, its own intermediate completion handler, and its own lifetime management. The composition worked. It was deployed. It scaled from low-level to high-level. But it was genuinely difficult to write, difficult to review, and difficult to get right. The authors wrote those state machines and can attest to the cost.

P2464R0[1] was right that this was ad-hoc. Senders provide a generic composition algebra that the Networking TS did not have. That is a real achievement.

Coroutines address the same axis differently. `co_await` replaces the state machine. A composed read that required a hundred-line state machine in the callback model is a ten-line coroutine body. The coroutine frame holds the state. The compiler manages the suspension points. C++20 was ratified in 2020.

6.2 A Second Axis

The preceding subsection addresses algorithmic composition - the axis P2464R0[1] was right about. P2464R0[1]'s composition argument also touched a second, distinct axis - the completion-model axis:

> *"How many different implementations of this function do I need to support 200 different execution strategies and 10,500 different continuation-decorations(*)? ... With Senders and Receivers, the answer is 'one, you just wrote it.'"*

"Execution strategies" means different executors - thread pools, event loops, strands, GPU contexts. The axis of ***where*** the continuation resumes. "Continuation-decorations" means different completion models - callbacks, futures, coroutines, `use_awaitable`. The axis of ***how*** the initiator receives the result.

On this axis, a solution existed. N3747[11], "A Universal Model for Asynchronous Operations" (Kohlhoff, 2013), documented the mechanism: `async_result`. Each completion token specializes `async_result` once, and every async operation works with it. One function signature serves every completion model:

```
template<class ReadHandler>
DEDUCED async_read(
    AsyncReadStream& s,
    DynamicBuffer& b,
    ReadHandler&& handler);
```

The same `async_read` works with a callback, a future, or a coroutine. M executors and N completion models require one implementation of `async_read` plus N specializations of `async_result`. Not M x N. `async_result` is not algorithmic composition - it does not give you `retry` or `when_all`. It is completion-model

polymorphism: one implementation of each operation, multiple ways to receive the result. P2464R0[1] did not address `async_result` or N3747[11].

The compositions existed. Asio's own `async_read` composes `async_read_some`. Boost.Beast composes socket-level reads into HTTP message parsing into WebSocket frames - three layers, all using `async_result`, all scaling from low-level to high-level. Both were deployed before P2464R0[1] was written. P2464R0[1] said "I don't know" whether the TS's asynchronous operations could compose. N3747[11] was published in 2013. Boost.Beast shipped in 2017.

6.3 The Two Framings in Code

P2464R0[1]'s composition argument rests on pseudo-code. Compare it with the code it was analyzing:

The continuation framing (P0113R0, 2015)

```
// Pass the result to the handler,
// via the associated executor.
dispatch(work.get_executor(),
    [line=std::move(line),
     handler=std::move(handler)]
        () mutable
    {
        handler(std::move(line));
    });
```

The work framing (P2464R0, 2021)

```
void run_that_io_operation(
    scheduler auto sched,
    sender_of<network_buffer> auto
        wrapping_continuation)
{
    launch_our_io();
    auto [buffer, status] =
        fetch_io_results_somehow(
            maybe_asynchronously);
    run_on(sched,
        queue_onto(wrapping_continuation,
            make_it_queueable(
                buffer, status)));
}
```

2015 and 2021.

6.4 Summary

On the algorithmic-composition axis, P2464R0[1] was right. The Networking TS had no generic composition mechanism. Senders provide one. Coroutines provide another. On the completion-model axis, a solution existed in N3747[11] and was deployed. P2464R0[1] did not address it.

The preceding sections address P2464R0[1]'s criteria. The following sections document additional properties of the coroutine constraint.

---

7. Structured Concurrency

The coroutine executor model provides structured concurrency through `co_await`-based combinators. The following examples are from Capy[4].

Fan-out with `when_all` - three concurrent operations, one join point:

```
capy::task<dashboard> load_dashboard(
    std::uint64_t user_id)
{
    auto [name, orders, balance] =
        co_await capy::when_all(
            fetch_user_name(user_id),
            fetch_order_count(user_id),
            fetch_account_balance(user_id));
    co_return dashboard{name, orders, balance};
}
```

Race with `when_any` - first completion wins, the other is cancelled:

```
capy::task<> timeout_a_worker()
{
    auto result = co_await capy::when_any(
        background_worker("worker"),
        capy::delay(100ms));

    if (result.index() == 1)
        std::cout << "Timeout fired\n";
}
```

---

8. The Executor Is Application Policy

P2464R0[1] treated thousands of executor implementations as a problem:

> *"C++ programmers will write **thousands** of these executors."*

P0113R0[10] (Kohlhoff, 2015):

> *"Like allocators, library users can develop custom executor types to implement their own rules."*

P0285R0[12] (Kohlhoff, 2016):

> *"The central concept of this library is the executor as a policy."*

The coroutine executor concept (Capy[4]):

```
template<class E>
concept Executor =
    std::is_nothrow_copy_constructible_v<E> &&
    std::is_nothrow_move_constructible_v<E> &&
    requires(E const& ce, std::coroutine_handle<> h)
{
    { ce == ce } noexcept -> std::convertible_to<bool>;
    { ce.context() } noexcept;
    { ce.dispatch(h) } -> std::same_as<
        std::coroutine_handle<>>;
    { ce.post(h) };
};
```

Two operations. Both take a `coroutine_handle<>`. Both resume it on a context. Most users never see it - they write `task<>` coroutines and pick an executor at the launch site. The concept exists for the people who build execution contexts, not the people who use them.

The narrowness of the concept limits the implementation space. `execute(F&&)` accepts any callable, so every executor implementation decides independently what to do with an arbitrary callable - error handling, lifecycle, composition are all implementation-defined. `dispatch(coroutine_handle<>)` and `post(coroutine_handle<>)` accept a type with exactly two operations: `resume()` and `destroy()`. A thread pool, an event loop, and a strand all implement the same two operations the same way. The variation is in ***where*** the handle resumes, not in ***what*** the executor does with it. Thousands of coroutine executors would differ only in scheduling policy. The ecosystem-scale divergence that P2464R0[1] predicted for `execute(F&&)` does not arise when the argument type is constrained.

9. Five Years Later

P2300R10[6] provides compile-time sender composition, structured concurrency guarantees, and a customization point model that enables heterogeneous dispatch. These are real achievements. P2464R0[1] identified real deficiencies in P0443R14[3] and the committee acted on them. Five years of outcomes are now observable.

9.1 The Empirical Record

Criterion	P2464R0[1] claim (2021)	Predicted outcome	2026 evidence
Error channel	`execute(F&&)` has no error channel	P2300R10[6] provides `set_error`	P2430R0[8] (Kohlhoff, 2021): compound I/O results cannot use `set_error` without losing the byte count. P2762R2[9] (Kühl, 2023): five routing options documented, single-argument error channel "somewhat limiting." LEWG reflector (March 2026): no standard facility exists for runtime dispatch of compound results onto channels.
Lifecycle	`execute(F&&)` has no lifecycle for submitted work	P2300R10[6] provides structured lifecycle via sender/receiver	P3801R0[13] (2025): `execution::task` lacks symmetric transfer. P3552R3[14]: `AS-EXCEPT-PTR` converts routine `error_code` to `exception_ptr`.
Generic composition	No generic composition	P2300R10[6] provides sender algorithms	P2470R0[15] (2021): deployments at Facebook, NVIDIA, Bloomberg - GPU dispatch, thread pools, infrastructure. Seven published examples (2024): thread pools, embedded systems, cooperative multitasking, custom algorithms. None involve networking. None involve I/O.
Deployed networking	"I don't know" whether Networking TS compositions scale	P2300R10[6] replaces the Networking TS	No published paper documents production-scale networking using the sender model. N1925[16] (2005): first networking proposal. 2026: networking is not in the C++ standard.

9.2 The Symmetry Test

A constraint applied to one model applies to both.

Ecosystem-scale validation:

Property	Coroutine executor	P2300R10[6] for networking
Age	P4003R0[2] (2026). New.	P2464R0[1] redirected the committee toward P2300[6] in 2021. Five years.
Deployments	Capy[4], Corosio[5].	P2470R0[15]: Facebook, NVIDIA, Bloomberg - GPU dispatch, thread pools, infrastructure.
Networking deployments	New.	None published. Boost.Asio and Boost.Beast - the deployed networking that the committee set aside - used the continuation model.

The `noexcept` context. Both models can reach `std::terminate` in a `noexcept` coroutine. The difference is in the trigger condition:

Property	Coroutine executor	`execution::task` (P3552R3[14])
What throws	`post(coroutine_handle<>)` throws `std::system_error` on scheduling failure.	`AS-EXCEPT-PTR` converts routine `error_code` to `exception_ptr`.
Trigger condition	Scheduling failure on an I/O reactor.	`ECONNRESET`, `ETIMEDOUT`, `EWOULDBLOCK` - routine I/O outcomes.
In a `noexcept` context	`std::terminate` on catastrophic system condition.	`std::terminate` on routine I/O.

Process and outcomes. The committee evaluates specifications as written. The committee also exists to deliver a standard that serves users. Both are true. This paper documents outcomes. The Networking TS was published as an ISO TS in 2018. It was removed from the committee's work program in 2021. In 2026, no replacement has shipped.

9.3 Summary

Criterion	P2464R0[1] (2021)	P2300R10[6] (2026)	Coroutine executor
Error channel	`execute(F&&)` has none	`set_error` exists; does not handle compound I/O results without information loss (P2430R0[8])	Not needed; result delivered to continuation on resume
Lifecycle	`execute(F&&)` has none	Structured lifecycle exists; `task` converts routine errors to exceptions (P3552R3[14])	Ownership contract: resume or destroy
Generic composition	`execute(F&&)` has none	Sender algorithms exist; deployed for GPU dispatch, thread pools, infrastructure	Algorithmic: `co_await` replaces state machines. Completion-model: `async_result` (N3747[11])
Deployed networking	"I don't know"	None published	New. Boost.Asio and Boost.Beast used the continuation model.

The question that P2464R0[1] asked in 2021 - whether the executor abstraction is adequate for async programming - now has two published answers instead of one.

Acknowledgments

The authors thank Peter Dimov for identifying that P0443R14's callable destruction is detectable, correcting an earlier version of Section 4; Dietmar Kühl for `beman::execution` and ongoing work on P3552R3; Ville Voutilainen for P2464R0, which provided the evaluation framework for this paper; Steve Gerbino and Mungo Gill for Capy and Corosio implementation work; and Klemens Morgenstern for Boost.Cobalt and the cross-library bridge examples.

References

1. P2464R0 - "Ruminations on networking and executors" (Ville Voutilainen, 2021). https://wg21.link/p2464r0
2. P4003R0 - "Coroutines for I/O" (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). https://wg21.link/p4003r0

3. P0443R14 - ''A Unified Executors Proposal for C++'' (Jared Hoberock, et al., 2020). https://wg21.link/p0443r14
4. Capy - Coroutine I/O foundation library (Vinnie Falco). https://github.com/cppalliance/capy
5. Corosio - Coroutine networking library (Vinnie Falco). https://github.com/cppalliance/corosio
6. P2300R10 - ''std::execution'' (Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach, 2024). https://wg21.link/p2300r10
7. P4094R0 - ''Retrospective: The Unification of Executors and P0443'' (Vinnie Falco, 2026). https://wg21.link/p4094r0
8. P2430R0 - ''Partial success scenarios with sender/receiver'' (Christopher Kohlhoff, 2021). https://wg21.link/p2430r0
9. P2762R2 - ''Sender/Receiver for Networking'' (Dietmar Kühl, 2023). https://wg21.link/p2762r2
10. P0113R0 - ''Executors and Asynchronous Operations, Revision 2'' (Christopher Kohlhoff, 2015). https://wg21.link/p0113r0
11. N3747 - ''A Universal Model for Asynchronous Operations'' (Christopher Kohlhoff, 2013). https://wg21.link/n3747
12. P0285R0 - ''Using customization points to unify executors'' (Christopher Kohlhoff, 2016). https://wg21.link/p0285r0
13. P3801R0 - ''Symmetric Transfer for task'' (Maikel Nadolski, 2025). https://wg21.link/p3801r0
14. P3552R3 - ''Add a Coroutine Task Type'' (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
15. P2470R0 - ''Slides for presentation of P2300R2'' (Eric Niebler, 2021). https://wg21.link/p2470r0
16. N1925 - ''A Proposal to Add Networking Utilities to the C++ Standard Library'' (Chris Kohlhoff, 2005). https://wg21.link/n1925